

ELI V2.0 · FINE-TUNING

Fine-tuning ELI (LoRA)

ELI can improve its own weights on your data using LoRA — a lightweight, opt-in fine-tuning path that runs locally, overnight, and never blocks your day. This guide covers what it does, what it needs, and how to run it safely.

Edition: v2.0.16 · **Audience:** advanced users and tinkerers · **Author:** ShadowESC95

Status: heavy, opt-in feature — *not* part of the default install. It's for people who want ELI to learn their domain.

What LoRA does, and why it's opt-in

LoRA (Low-Rank Adaptation) trains a small set of adapter weights on top of a frozen base model, rather than retraining the whole thing. It's the practical way to teach a local model your style, your domain, or your corrections — with modest hardware and time.

Because it pulls in a heavy stack (PyTorch, Transformers, PEFT, TRL, Datasets), LoRA is **deliberately excluded from the standard install**. You enable it only if you want it. Everything else in ELI runs without it.

Honest expectations

Fine-tuning is powerful and finicky. Results depend on your data quality, your base model, and your GPU.

Treat it as a capable experimental feature, not a one-click miracle. Start with a dry run, review everything, and keep backups.

1 • Install the training stack

The training dependencies are a separate optional group so they never bloat a normal install:

```
.venv/bin/python -m pip install -e ".[training]"  
# torch, transformers, peft, datasets, trl
```

A CUDA-capable NVIDIA GPU is strongly recommended. CPU training is possible in principle but impractically slow for real work.

2 • Check readiness first (preflight)

Before training anything, ask ELI whether it's actually ready. The preflight checks the three things that break a run: the required modules, a usable base model, and reviewed training data.

```
is lora ready          # or: "lora status"
```

ELI reports each check (modules present · base model resolved · data reviewed) so you fix problems before burning GPU hours, not after.

3 • The training pipeline

Fine-tuning runs as a real, staged pipeline on ELI's DAG orchestrator — each stage gated on the previous one:

Stage	What happens
-------	--------------

Preflight	Verifies modules, base model, and reviewed data. Aborts early with a clear reason if anything's missing.
Build	Assembles the training dataset from your reviewed examples into the trainable format.
Train	Runs the LoRA training loop against the frozen base model, producing adapter weights.
Eval	Evaluates the result so you can see whether the adapter actually improved anything.

4 • Dry-run from chat, real training overnight

ELI separates *thinking about* training from *doing* it, so a heavy job never freezes your session:

- **From chat** — "fine-tune yourself" / "train a lora" performs a **dry run**: it plans and validates the pipeline without committing GPU hours.
- **The real run** is scheduled as an **overnight task** (see the Tasks tab). It runs in the background, survives restarts, and reports when done — so you kick it off and get your machine back.

```
train a lora overnight      # schedules the real preflight→build→train→eval run
```

5 • Data: reviewed, yours, local

The training data is built from examples ELI has gathered and that you've **reviewed** — the preflight will refuse to train on unreviewed data. Everything stays on your machine; nothing about your fine-tune is uploaded. This is the same privacy posture as the rest of ELI: local or nothing.

6 • Keeping ELI honest over time

Independently of manual fine-tunes, ELI runs standing nightly jobs — an engine evaluation plus a test report, and test generation — so its own quality is measured continuously rather than assumed. You can inspect these from the Tasks and Test & Review surfaces.

Troubleshooting

Symptom	Fix
"lora not ready" — modules missing	Install the training extra: <code>pip install -e ".[training]"</code> .
Preflight fails on the base model	Make sure a compatible base model is present and resolvable; ask ELI "lora status" for the exact path it checked.
Preflight fails on data	

You have unreviewed training data — review it first; the pipeline won't train on unvetted examples.

Out of VRAM during train

Use a smaller base model or reduce the batch/sequence length; LoRA is light but the base model still has to fit.

ELI v2.0 — local, opt-in LoRA fine-tuning. Preflight → build → train → eval, overnight, on your data, on your hardware. For the everyday feature set see the *User Manual*.